14.10 Characteristics of Constructors

- The name of the constructor must be same as that of the class
- No return type can be specified for constructor
- A constructor can have parameter list
- The constructor function can be overloaded
- They cannot be inherited but a derived class can call the base class constructor
- The compiler generates a constructor, in the absence of a user defined constructor.
- Compiler generated constructor is public member function
- The constructor is executed automatically when the object is created
- A constructor can be used explicitly to create new object of its class type

14.11 Destructors

When a class object goes out of scope, a special function called the *destructor* gets executed. The destructor has the same name as the class tag but prefixed with a **~(tilde)**. Destructor function also return nothing and it does not associated with anydata type.

14.11.1 Need of Destructors

The purpose of the destructor is to free the resources that the object may have acquired during its lifetime. A destructor function removes the memory of an object which was allocated by the constructor at the time of creating a object.

14.11.2 Declaration and Definition

A *destructor* is a special member function that is called when the lifetime of an object ends and destroys the object constructed by the constructor. Normally declared under public.

Illustration14.8 To illustrate destructor function in a class

```
#include<iostream>
using namespace std;
class simple
{
private:
int a, b;
public:
simple()
{
a= 0 ;
b= 0;
cout<< "\n Constructor of class-simple ";
}
void getdata()
{
cout<<"\n Enter values for a and b ";
cin>>a>>b;
}
void putdata()
{
cout<<"\nThe two integers are .. ";
cout<<<<a<<'\t'<< b<<endl;
cout<<"\n The sum = "<<a+b;
}
~simple()
{    cout<<"\n Destructor is executed ";}
};
int main()
{
simple s;
s.getdata();
s.putdata();
return 0;
}
```

Output:

```
Constructor of class-simple
Enter values for a and b  6 7
The two integers are .. 6      7
The sum = 13
Destructor is executed
```

14.12 Characteristics of Destructors

- The destructor has the same name as that class prefixed by the tilde character '~'.
- The destructor cannot have arguments
- It has no return type
- Destructors cannot be overloaded
- In the absence of user defined destructor, it is generated by the compiler
- The destructor is executed automatically when the control reaches the end of class scope to destroy the object
- They cannot be inherited

Chapter 14 Page 233-248.indd 244 3/24/2020 9:23:04 AM